R SO Cheatsheet

1. Stiva software

Apel de sistem

Definitie: mod de comunicare intre un program si sistemul de operare

Necesitate:

- Securitate – protejeaza hardware-ul de aplicatii malitioase
- Portabilitate – ofera o interfata constanta pentru majoritatea hardware-urilor
- Abstractizare – aplicatiile nu au nevoie sa stie detalii despre hardware
- Performanta – optimizarea folosirii resurselor

Avantaje:

Mentionate mai sus

Dezavantaje:

- Overhead – schimabre intre user si kernel mode
- Complexitate – parametrii return-uri si erori specifice
- Securitate – bug-uri in implementare pot cauza exploatari
- Functionalitate limitata
- Intreruperi

Apel de sistem blocant = blocheaza procesul pana serviciul este completat

Ex: read(), wait()

Apel de sistem neblocant = nu blocheaza sistemul ci doar notifica daca call-ul a reusit sau nu.

Ex: poll(), fcntl()

User space vs kernel space

Definitie**:**

user space = aplicatiile software si unele drivere; neprivilegiat; flexibil si divers

kernel space = rezervat pentru kernelul os-ului, extensii ale acestuia si drivere; este privilegiat

Tranzitia in modul privilegiat se face prin intermediul apelurilor de sistem(bitul de mod este setat la 0 inainte de trecerea in kernel mode si la 1 inainte de trecerea in user mode) si este de obicei facute de catre CPU.

SO se asigura de rularea numai a codului autorizat prin folosirea anumitor mecanisme de control precum: ACL(Access control list/database), paginare, segmentare.

Actiuni kernel space:

- access la resurse I/O
- alocari de memorie
- comunicare intre aplicatii

Actiuni user space:

- access la memorie
- operatii logice si aritmetice
- apeluri de functii

ISA = interfata oferita software-ului pentru a comunica cu hardware-ul

ex: x86, ARM, RISC

Kernel space ISAs:

- LGDT, LLDT, LTR, and LIDT: load the global descriptor table (GDT), local descriptor table (LDT), task register (TR), and interrupt descriptor table (IDT) - memory management

- CLI and STI : enable and disable interrupts – execution flow control

- IN and OUT: read from and write to I/O - communication

- RDMSR and WRMSR: read from and write to model-specific registers (MSRs) - CPU communication

- SMSW: read the value of the machine status word (MSW) - CPU current state

- + paging, segmentation, and control registers

Biblioteca

Definitie: o colectie de resurse nevolatile folosite de aplicatii/ o interfata oferita programelor pentru a indeplini anumite task-uri

Apel de sistem vs apel de biblioteca:

Apelurile de biblioteca se pot folosi de apelurile de sistem pentru a comunica cu hardware-ul.(nu e obligatoriu)

Apelurile de biblioteca invoca functii in user space => sunt mai rapide

Libc - o colectie de biblioteci care ofera functionalitati de baza precum: input/output, alocare d ememorie, manipularea string-urilor etc.

- ofera functii si macro-uri frecvent folosite de programele C

Biblioteca statica vs dinamica:

Definitie: *Biblioteca statica =* o colectie de obicete combinate intr-un singur fisier. Sunt pre-compilate, self-contained(un executabil linkat contine tot codul librariei), depinde de platforma si executabile ocupa mai mult spatiu.

Biblioteca dinamica = o colectie de obiecte care sunt separate de executabilele care o folosesc. Sunt incarcate la runtime deci nu sunt precompilate. Sunt independente de platforma si executabilele nu ocupa la fel de mult spatiu.

Sistem de operare monolitilc vs microkernel

Definitie: *SO monolitic* = arhitectura in care intreg sistemul de operare functioneaza ca un singur process in kernel space. Cel mai comun. Defineste o interfata virtuala high level asupra hardware-ului

So microkernel = kernelul este spart in mai multe parti care ofera doar servicii de baza precum low-level address space management, thread management si comunicare intre procese(IPC). Drivere si sistemuld e fisiere dus in user-space.

Avantaje:

-Monolith:

Performanta - mai rapid deoarece se face un singur apel de sistem pentru realizarea unui serviciu

Resource management eficient – access direct la memorie si CPU

Backward compatibility mai bun – pot rula pe hardware mai vechi

-*Microkernel:*

Modularitate – izolarea bug-urilor sau a vulnerabilitatilor

Poratbilitate – core-ul kernelului este mic si simplu si poate fi portat rapid pe un alt hardware

Flexibilitate – servicii de sistem pot fi usor adaugate, sterse sau modificate fara a afecta restul sistemului

Securitate – serviciile ruleaza in user space

Scalabilitate – poate rula pe mai multe procesoare intr-un sistem distribuit

Monolithul are mai multe apeluri de sistem datorita faptului ca toate serviciile sunt oferite de un singur proces si au access la toate datele.

Kernelul unui so ce foloseste arhitectura de microkernel are ai putine responsabilitati => mai putine syscalluri

Masini Virtuale - environment creat de un hipervizor(middleman)

Avantaje securitate:

Izolare: adauga un strat de protectie intre os-ul gazda si os-ul folosit de masina virtuala.

Segregare: pot fi folosite mai multe masini virtuale independente pentru a realiza un task, astfel daca una dintre ele este compromisa, aceasta nu va compromite intreg sistemul

Sandboxing: pot fi foloste pentru a crea un spatiu de rulare a programelor posibil malitioase fara a compormite os-ul gazda

Snapshoting: starea masinii virtuale poate fi salvata si reincarcata la pornire.

Overhead spatial - spatiu sau memorie aditionala necesara organizarii si management-ului de date dintr-un sistem

Tipuri:

- Memory overhead – memorie aditionala pentru pastrarea unor structuri de date esentiale precum tabele de alocare si dealocare
- Disk overhead – spatiu pe disc alocat structurilor de date precum tabelele de alocare pentru fisiere
- Network overhead – bandwith-ul si spatiul aditional necesar pentru management-ul de transfer de date peste o retea
- Data overhead – datele aditionale pentru a pastra meta-datele cum ar fi timestamp-uri sau numerele versiunilor

Overhead temporal – timp aditional pentru a realiza un anumit task intr-un sistem

Tipuri:

- CPU overhead – timpul aditional pentru organizarea datelor intr-un sistem precum alocarea si dealocare de spatiu
- I/O overhead – timpul aditional necesar realizarii operatiilor de input/ output si timpul de asteptare pentru completarea lor
- Syncronization overhead – timpul aditional necesar sincronizarii acesului la resurse partajate precum semafoare sau mutexuri.
- Scheduling overhead – timpul aditional necesar planificarii si schimbarii intre procese si thread-uri
- Data overhead =- timpul necesar procesraii meta-datelor

Double buffering – tehnica prin care se folosesc doua buffere alternativ pentru a scrie si citit date

Usecase: In optimizarea operatiilor I/O cu sistemul de fisiere. Unul dintre buffere mentine starea curenta a datelor de pe disck si unul pentru starea updatata.

In reducerea overhead-ului cauzat de context switching.

Seg fault => terminarea programului si creaarea unui core dump(fisier ce contine o imagine a memoriei programului inainte de terminare). Sistemul de operare trebuie sa trateze eroarea.

2. Scheduling(calcul)

Process = instanta unui program executat de so. Un container ce contine toate rsursele necesare executarii: memorie, resurse de sistem si fisiere deschise

Thread = lightweight process. A separate flow of execution(nu stiu cum se traduce). UN proces poate avea mai multe thred-uri.

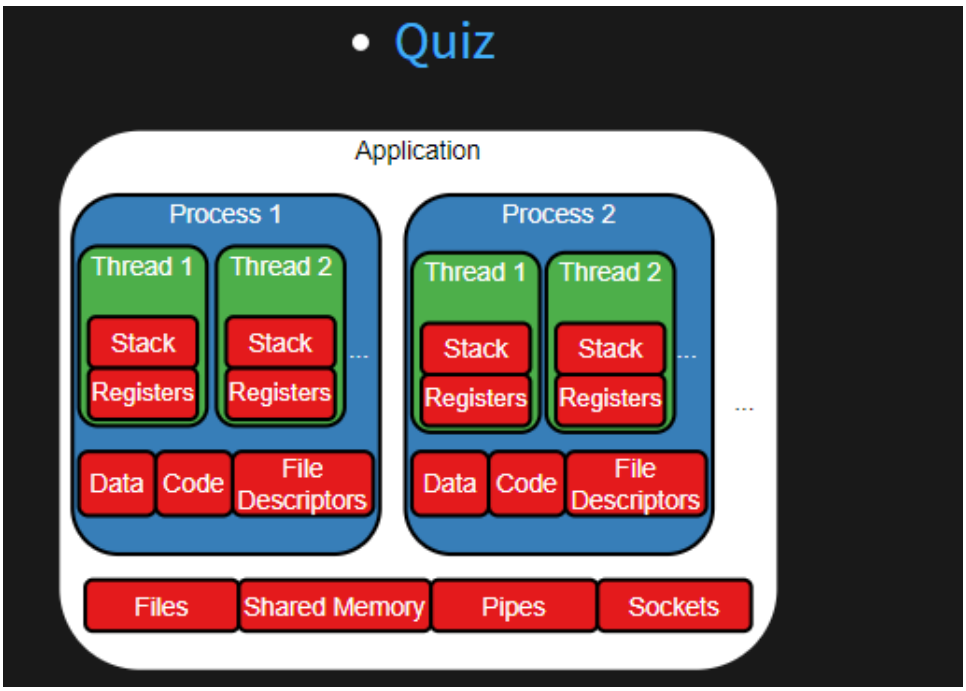
Diferente:

- Memory space
- Resurse
- Scheduling
- Izolare
- Comunicare

- Start-up time

PROCESS VS THREAD	
PROCESS	**THREAD**
independent	part of a process
isolated VAS	shares VAS with other threads
slower creation	faster creation
larger context switch duration	shorter context switch duration
ending means ending all threads	other threads continue when finished

VAS: Un thread share-uieste spatiul de memorie cu procesul ce l-a creat. Acesta are acces la codul si datele procesului si primeste propriile registre si stack.



PCB (process control block) = structura de date folosita pentru a tine informatii despre procese:

- PID
- Starea procesului
- Registrii CPU
- Alocare memorie
- Informatii despre planificarea CPU
- I/O status

Stari proces/thread:

- Running - este in parcurs de executare
- Waiting – asteapta un event cum ar fi o operatie I/O, pentru deblocarea semaforului
- Sleeping - este suspendat temporar de catre so sau proces
- Ready – este gata de rulare
- Terminated – si-a terminat executia
- Zombie – a fost terminat insa procesul parinte nu este inca constient de asta

Fork() = apel de sistem ce creeaza un nou proces

Procesul copil are propriul PID, Adress Space si resurse de sistem

Au in comun codul in sine, ambele executand codul de dupa fork()

Exec() = apel de sistem ce inlocuieste procesul curent cu unul complet nou. Isi pastreaza PID-ul.

Context switch = procesul de a salva starea curenta a unui proces si restaurarea starii unui altuia pentru a lasa mai multe procese sa fie executate de catre CPU simultan. Este facut de scheduler.

Cauzat de: I/O requesting, asteptarea unui semafor, blocarea procesului, terminarea quantei de timp a procesului, aparitia unui proces cu prioritate mai mare

Schimbare de context voluntara : un proces preda voluntar locul pe CPU. Cauzat de operatii I/O sau semafoare.

Schimbare de context involuntara : cand un proces este preemptat de scheduler. Cauzat de aparitia unui proces cu prioritate mai mare sau de terminarea quantei de timp. Poate fi cauzat de un hardware interrupt pentru a face loc pentru handler.

User level threads vs kernel level threads :

User level threads sunt threadurile de care se ocupa aplicatia. Exista doar la nivelul aplicatiei si nu in so. Pot purta nume prcum corutine, fibre etc. Previn race conditionuri si sunt mai simplu de implementat. Acestea se pot regasi si ca “Green threads”.

Kernel level threads/OS-level threads/native threads sunt thredurile create de so si pot folosi mai multe core-uri ale procesorului. Sunt mai greu de implementat in aplicatii.

TLS(Thread local storage) = o zona de stocare care permite unui thread sa aiba propria copie a unui obiect, in loc sa il share-uiasca cu alte threaduri.

Cazuri de folosinta:

Cand o aplicatie are nevoie sa retina date specifice despre threaduri cum ar fi coduri de eroare sau buffere specifice

Cand un thread are nevoie sa tina minte date soecifice lui cum ar fi fisierele deschise sau conexiuni la retea

Cel mai folosit: pentru numarare statistica. Un counter este impartit mai multor threaduri fiecare facand update prorpiului contor, princpipalul contor fiind suma tutror.

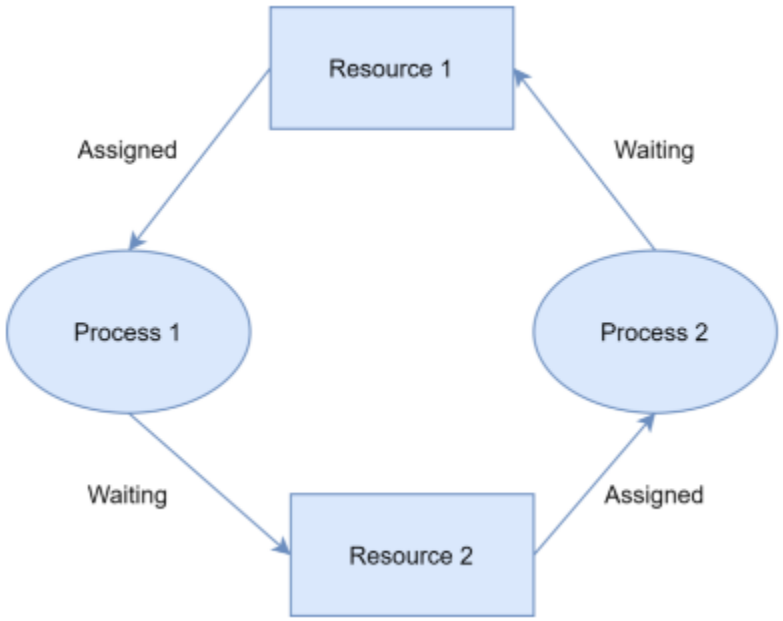
Sincronizarea threadurilor

Necesara pentru ca threadurile sa nu intre unul peste celalalt in momentul modificarii datelor comune. Fara sincronizare, scrierea datelor poate duce la inconsistente, race conditionuri si multe alte erori.

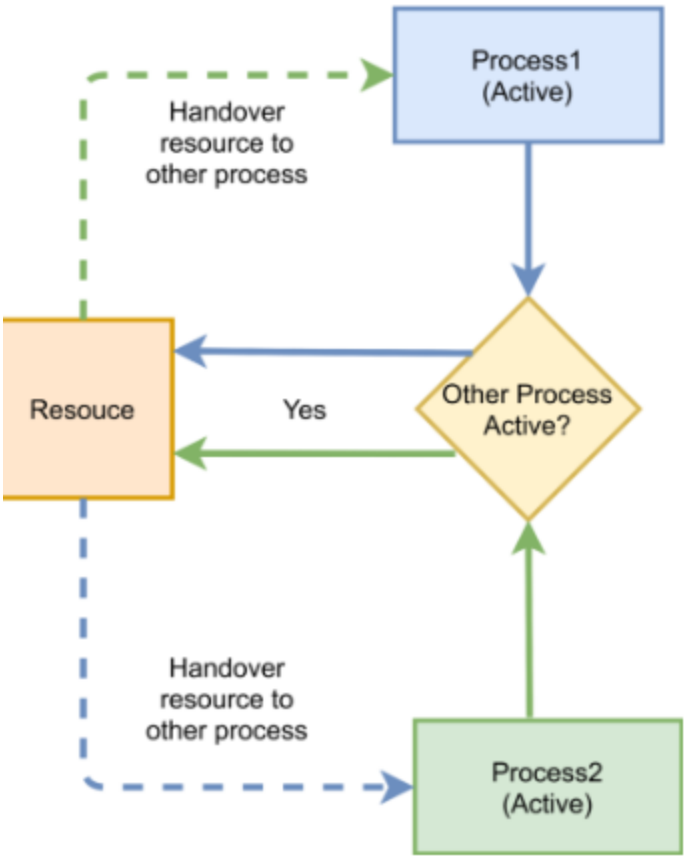
Dezavantaje: impact asupra performantei, deadlocks, complexitatea codului, inversarea prioritatii.

Race condition: un tip de eroare cauzat de doua threaduri ce aceseaza un obiect share-uit in acelasi timp incercand s ail modifice.

Deadlock: un tip de eroare cauzat in momentul in care doua threaduri retin resurse si se asteapta unul pe altul la eliberarea resurselor. Poate cauza ca threadurile sa fie blocate la infinit si blocheaza aplicatia.



Livelock: ca un deadlock insa threadurile incearca in mod constant sa obtina o singura resursa, astfel aplicatia nu se blocheaza insa nu face progrese



Toctu(time of check to time of use) = vulnerabilitate de securitate care are loc cand un program verifica o resursa insa o foloseste mai tarziu, cand aceasta si-a schimbat continutul.

Problema producator-consumator = problema ce decrie un scenariu in care unul sau mai multe threaduri(producatori) produc obiecte si le salveaza intr-un buffer comun de unde alte threaduri (consumatori) le pot folosi.

Spinlock = mecanism de sincornizare in care un thread verifica constant starea lock-ului intr-o bucla

Mutex = spinlock insa threadul se blocheaza pana la eliberarea lockului

1. Spinlock is a type of lock that causes a thread attempting to obtain it to check for its availability while waiting in a loop continuously. other hand, a mutex is a program object designed to allow different processes may take turns sharing the same resource.
2. The spinlock temporarily prevents a thread from moving. In contrast, the mutex may block a thread for an extended amount of time.
3. Spinlock is useful for limited critical sections; else, it wastes the CPU cycles. On the other hand, the mutex is useful for crucial extende where frequent context switching would add overhead.
4. Spinlock makes no use of context switching. In contrast, mutex involves context switching.
5. The process in spinlock may not sleep while waiting for the lock. In contrast, the process in mutex may sleep while waiting for the lock.
6. Spinlock disables preemption. On the other hand, mutex supports preemption.

The '&' operator = face ca procseul sa ruleze in background

Zombie process = un proces care si-a completat executia dar inca exista in tabela de procese. Se intampla pentru procesel copil, pentru care este nevoie pastrarea in tabela pentru ca procesul aprinte sa poata citi starea de iesire (exit status).

Ocupa resurse degaba daca nu sunt eliminate.

Orphan process = proces al carui parinte si-a terminat executia insa el inca ruleaza. Un proces orfan este imdeiat adoptat de "init" si eliminat.

Thread stack sharing = un thread poate accesa stack-ul unui alt thread daca stie adresa, deoarece sunt in acelasi VAS si nu va cauza un seg fault.

Nr maxin de threaduri = */proc/sys/kernel/threads-max*.

number of threads = total virtual memory / (stack size*1024*1024)

Nr maxim de procese = `/proc/sys/kernel/pid_max`

Waiting time = timpul total petrecut de un process in starea ready

Putem avea un sistem multi-core cu un singur proces aflat în starea RUNNING și mai multe procese în READY?

Da, daca procesul are mai multe threaduri si prioritatea cea mai mare.

I/O intensive = multe operatii cu fisiere

Ex:

- Database management
- Media processing
- Network apps
- Back-up/restore

CPU intensive = multe calcule

Ex:

- Simulari
- Cryptografie
- Machine learning
- Video encoding/decoding
- Image processing

Apeluri blocante intr-un spilock : poate duce la livelock

Wait() - proces

Join() - threaduri

3. Date

Separare procese – VAS

Memorie virtuala = abstractizare a memoriei reale a sistemului. Se realizeaza prin maparea adreselor virtuale vazute de proces la adrese reale din RAM

VAS(Virtual address space) = un interval de adrese pe care poate sa le foloseasca un proces pentru a ajunge la memoria reala. Sunt mapate de MMU. Sunt impartite in pagini de marime fixe(de obicei 4 kB).

Zone de memorie:

- .text/code - read, execute
- .data - variabile statice si globale, read-write
- .rodata - variabile constante, read-only
- .bss(block started by symbol) - variabile neinitializate, read-write
- Stiva(stack) - variabile locale, frame-uri de functii, read-write
- Heap – memorie alocata dinamic, read-write

Paginarea memoriei = modalitate de a stoca date in bucati de marime fixa si de a le transfera din memoria principala(RAM0 in cea secundara (disk)

Tabela de pagini = structura de date ce retine maparile dintre memoria virtuala si cea reala.

MMU(memory management unit) = hardware ce se ocupa de translatarea adreselot virtuale in adrese fizice.

4. I/O

FCB(File control block) = structura de date care contine starea unui fisier deschis. Este in memoria prograului care foloseste fisierul.

Offset	Byte size	Contents
0x00	1	Drive number — 0 for default, 1 for A:, 2 for B:, ...
0x01	8	File name and extension — together these form a 8.3 file name.
0x09	3	
0x0C	20	Implementation dependent — should be initialised to zero before the FCB is opened.
0x20	1	Record number in the current section of the file — used when performing sequential access.
0x21	3	Record number to use when performing random access.

The 20-byte-long field starting at offset 0x0C contained fields which (among others) provided further information about the file:[2]

Offset	Byte size	Contents
0x0E	2	File's record length in bytes.
0x10	4	Total file size in bytes.
0x14	2	Date of last modification to file contents.
0x16	2	Time of last modification.

File descriptor = un numar intreg ce identifica un fisier deschis intr-un sistem de operare.

Tabel de file descriptori = vector asociativ intre fd si referinta la fisier. Unul pentru fiecare proces

Dup() - creeaza o copie a fd-ului luand cel mai mic fd nefolosit. Ambele fd-uri referentieaza acelasi fisier.

Close() - inchide un fd astfel incat sa nu mai referentieze nici-un fisier deschis.

Daca e ultimul fd catre fisier, toate resursele se elibereaza.

Fseek(), lseek() - seteaza pointer-ul intr-un fisier.

Truncate() - modificare marime fisier

Tipuri de fisiere : *regular*, *directory*, *symbolic link*, *FIFO special(pipe)*, *block special*, *character special*, and *socket*

VFS(Virtual file system/ virtual file system switch) = program ce ofera o interfata pentru a lucra cu un sistem de fisiere.

Dispozitiv virtual = un device file care nu are asociat un hardware. Creat cu mknod().

Tipuri de dispozitive in UNIX:

Block-oriented – transfera datele in blocuri(ex hard disk); de obicei blocurile au 1KB

Character-oriented – transfera datele byte cu byte.(ex modemurile, dumb termminals, keyboards)

Socket() = structura de date definita de un API menita pentru comunicarea intre procese.

Accept() = apel de sistem care extrage primul request de conexiune din coada de conexiuni in asteptare a socket-ului ascultator dat ca parametru, creeaza un nou socket conectat si intoarce un fd catre acel socket.

Adresa IP locala si port-ul ale socketului creat vor coincide cu socket-ul care asculta(cel dat ca parametru primul).

Double-buffering on sockets – fiecare socket are 2 buffere, unul pentru primirea datelor si unul pentru trimiterea lor.

Socket de retea vs UNIX scoket:

Comunicare:

Socket-urile de retea sunt folosite pentru a comunica intr-o retea LAN sau internet

Socket-urile UNIX sunt folosite pentru comunicarea intre procese

Adresa pe care o referentiaza:

Socket-urile de retea contin o combinatie unica intre un IP si un port

Socket-urile UNIX folosesc sistemul de fisiere.

Tipuri de comunicare:

Socket-urile de retea folosesc protocale precu TCP si UDP

Securitate si Performanta

UNIX > retea

Read()/write() - intoarce numarul de bytes cititi/scrisi sau –1 la eroare.

Operatii pe fisiere:

1. **Open**
2. **Read/Write**
3. **Seek**
4. **Truncate**
5. **Close**
6. **Rename**
7. **Delete**
8. **Copy**
9. **Move**

Marimea si cursorul unui fisier sunt tinute in metadatele sistemului de fisiere pe disk

Operatie asincrona = operatie care ruleaza independent de rularea programului principal si nu blocheaza executia acestuia. Se folosesc de callback pentru updatare. Intoarce un handler sau referinta pentru a urmari rularea.

Operatie neblocanta = operatie ce returneaza imediat, fara a se astepta finisarea acesteia. Intoarce controlul programului principal in timp ce operatia este efectuata. Returneaza un status code care determina succesul sau esecul operatiei.

Pipe = mecanism de comunicare intre procese unidirectional. Creat cu pipe() care intoarce doi fd, unul pentru scriere si unul pentru citire.

Pipe anonim = folosit pentru comunicarea intre parinte si copil.

Pipe cu nume = numit si FIFO, are nume si este folosit pentru comunicarea intre procese diferite.

Write() este de obicei facut asincron, datele fiind scrise intr-un buffer. Singura modalitate de a fi blocant este ca buffer-ul sa fie plin, iar SO-ul nu poate aloca mai multa memorie pentru acesta, ceea ce se intampla rar.

Read() poate bloca daca nu exista date de citit si va astpta pana acestea sunt incarcate. Daca este in modul neblocant va intoarce imediat valoarea –1 in acest caz.

Device diver = program care controleaza un anumit device. Ofera o interfata software pentru componentele hardware si face posibila utilizarea lor de catre SO fara sa trebuiasca sa stie neaparat detaliile

Zero-copy = operatie in care procesorul nu copiaza datele dintr-o zona a memoriei in alta.

One common zero-copy technique is called "zero-copy sendfile", which allows data to be transmitted directly from a file on disk to a network socket without being copied to an intermediate buffer. This eliminates the need to copy the data to a buffer before it is transmitted, saving time and resources.

Printf() - apel de librarie, complex, ofera interfata high-level

Write() - apel de sistem, low-level

Sync() - cauzeaza toate modificarile in asteptare ale metadatelor sistemului de fisiere sau cele cache-uite sa fie scrise pe disk

Ioctl() - apel de sistem face operatii cu device-uri. Poate prelua date specifice despre device. Da un request code unui device pentru a face diferite operatii.

Fwrite() - datele sunt scrise intr-un buffer temporar si permite programului sa acumuleze mai multe date intr-un buffer inainte de a face un apel de sistem.

Dup vs 2 opens – folosind dup, fd sunt oarecum conectati. Astfel, daca se face lseek pe unul se va muta si cel de-al doilea. La open, cei doi fd sunt complet individualizati. Cu open, se creeaza o noua structura in sistemul de fisiere.

La inceputul unui process, shell-ul deschide 3 fisiere: stdin, stdou, stderr.